

2-Mavzu: PyQt5 paketi va QtDesigner dasturi yordamida GUI dasturlarini yaratish.
6-mashg'ulot. PyQt da rasmlar va menyular.

O'quv savollari:

1. PyQt paketi yordamida rasm joylashtirish usullari.
2. Menu yaratish. Menu vidgetining xususiyatlari.

1. PyQt paketi yordamida rasm joylashtirish usullari.

№	Usul nomi	Qisqacha tavsif	Qachon ishlatiladi
1	QLabel + QPixmap	Eng oddiy usul, rasmni oynada ko'rsatadi	Statik rasm, fon yoki belgi
2	QPushButton / QAction + QIcon	Rasmni tugma yoki menyu belgisi sifatida ishlatadi	Tugmalar, menyular, toolbar
3	QGraphicsView + QGraphicsScene	Katta rasmlar, zoom/pan bilan ishlash uchun	Galereya, viewer dasturlari
4	QPainter	Rasmni chizish, o'zgartirish, effektlar berish uchun	Grafik dasturlar, filtrlash

1. QLabel + QPixmap (asosiy va eng sodda yo'l)

Rasmni ko'rsatishning eng oson usuli.

Misol:

```
from PyQt5 import QtWidgets, QtGui
import sys

app = QtWidgets.QApplication(sys.argv)
label = QtWidgets.QLabel()

pix = QtGui.QPixmap("cat.jpg") # rasm yuklanadi
label.setPixmap(pix)
label.setScaledContents(True) # rasm o'lchamga moslashadi
label.resize(300, 300)
label.show()

sys.exit(app.exec_())
```

Afzalligi: juda oson, kichik rasmlar uchun ideal.

Kamchiligi: foydalanuvchi zoom, harakatlantirish qila olmaydi.

2. QPushButton yoki QAction bilan rasm (ikonka)

Rasmni tugma yoki menyu belgisiga joylashtirish mumkin. Bunda QIcon ishlatiladi.

Misol:

```
from PyQt5 import QtWidgets, QtGui
import sys
```

```

app = QtWidgets.QApplication(sys.argv)

btn = QtWidgets.QPushButton("Profil")
btn.setIcon(QtGui.QIcon("user.png")) # rasmni ikonka sifatida qo'shamiz
btn.setIconSize(QtCore.QSize(32, 32)) # ikonka o'lchami
btn.show()

sys.exit(app.exec_())

```

Afzalligi: tugmalarni chiroyli qiladi, menyu va toolbarlar uchun qulay.

Kamchiligi: faqat kichik rasmlar (belgi sifatida) ishlatiladi.

3. QGraphicsView + QGraphicsScene

Bu usul katta rasmlar yoki ko'rish oynasi kerak bo'lganda ishlatiladi (masalan, zoom, surish, aylantirish).

Misol:

```

from PyQt5 import QtWidgets, QtGui, QtCore
import sys

app = QtWidgets.QApplication(sys.argv)

scene = QtWidgets.QGraphicsScene()
pix = QtGui.QPixmap("nature.jpg")
scene.addPixmap(pix)

view = QtWidgets.QGraphicsView(scene)
view.setRenderHint(QtGui.QPainter.Antialiasing)
view.fitInView(scene.sceneRect(), QtCore.Qt.KeepAspectRatio)
view.show()

sys.exit(app.exec_())

```

Afzalligi: rasmni kattalashtirish, harakatlantirish, aylantirish mumkin.

Kamchiligi: kod biroz murakkab, kichik loyihalar uchun ortiqcha.

4. QPainter yordamida rasm chizish yoki tahrirlash

Bu usulda rasm ustida chizish, effekt berish, matn yozish, rang o'zgartirish mumkin.

Misol:

```

from PyQt5 import QtGui

pix = QtGui.QPixmap("flower.jpg")
p = QtGui.QPainter(pix)

```

```
p.setPen(QtGui.QPen(QtGui.QColor("red"), 5))
p.drawRect(20, 20, 100, 100)
p.end()
pix.save("flower_new.jpg") # rasmni saqlaymiz
```

Afzalligi: chizish, filtrlash, tahrir qilish uchun qulay.

Kamchiligi: interfeysda ko'rsatish uchun qo'shimcha vidjet kerak bo'ladi.

Xulosa

PyQt da rasm bilan ishlashda:

- **Statik rasm** → `QLabel`
- **Ikonka yoki belgi** → `QIcon`
- **Zoom, pan, interaktiv ko'rish** → `QGraphicsView`
- **Tahrir, chizish, yozuv** → `QPainter`

Qisqacha nazariya (oddiy tilda)

PyQt5 da oynaga rasm joylash uchun asosan **QLabel** vidjetidan foydalaniladi.

1. **QLabel** — bu matn yoki rasm ko'rsatadigan vidjet.
2. Rasmni chiqarish uchun `QPixmap` obyektidan foydalanamiz.
3. Qo'llanish tartibi:
4. `label.setPixmap(QPixmap("rasm.png"))`
5. Rasmni o'lchamga moslashtirish uchun:
6. `label.setScaledContents(True)`
7. Rasm fayli ilova bilan bir joyda bo'lishi kerak yoki to'liq manzil ko'rsatiladi.
Masalan: `"C:/Users/User/Desktop/rasm.jpg"`

1-misol: QLabel ichida rasm ko'rsatish

Qt Designer'da:

1. Yangi **Main Window** oching.
2. Markazga `QLabel` qo'ying.
3. `objectName` → **label**
4. `scaledContents` → **True** qilib belgilang.
5. Faylni saqlang: **rasm_demo.ui**

Python kodi (main.py):

```
from PyQt5 import QtWidgets, QtGui, uic
import sys

class RasmApp(QtWidgets.QMainWindow):
    def __init__(self):
        super().__init__()
        uic.loadUi("rasm_demo.ui", self)
```

```

        self.label.setPixmap(QtGui.QPixmap("rasm.jpg")) # shu joyda rasm nomini
yozing

app = QtWidgets.QApplication(sys.argv)
oyna = RasmApp()
oyna.show()
sys.exit(app.exec_())

```

Natija: Oynada `rasm.jpg` chiqadi.

(`rasm.jpg` faylni shu papkaga joylashtiring.)

2-misol: Tugma bosilganda rasmni o‘zgartirish

Qt Designer’da:

1. Bitta `QLabel` qo‘ying (objectName: **label**)
2. Bitta `QPushButton` qo‘ying (objectName: **btnChange**, text: “Rasmni o‘zgartir”)
3. Faylni saqlang: **change_rasm.ui**

Python kodi:

```

from PyQt5 import QtWidgets, QtGui, uic
import sys

class Window(QtWidgets.QMainWindow):
    def __init__(self):
        super().__init__()
        uic.loadUi("change_rasm.ui", self)

        # Dastlabki rasm
        self.label.setPixmap(QtGui.QPixmap("rasm1.jpg"))
        self.label.setScaledContents(True)

        # Tugma bosilganda boshqa rasm chiqsin
        self.btnChange.clicked.connect(self.change_image)

    def change_image(self):
        self.label.setPixmap(QtGui.QPixmap("rasm2.jpg"))

app = QtWidgets.QApplication(sys.argv)
w = Window()
w.show()
sys.exit(app.exec_())

```

Natija: Dastlab `rasm1.jpg` chiqadi, tugma bosilganda `rasm2.jpg` paydo bo‘ladi

2. Menu yaratish. Menu vidgetining xususiyatlari.

1) Menu nima?

Menu — GUI (grafik interfeys) da foydalanuvchiga buyruqlarni tartibli ko‘rinishda taklif qiladigan komponent. Odatda oynaning yuqorisidagi **Menu Bar** ichida joylashadi (masalan: *File, Edit, Help*).

2) Menu anatomiyasi

- **Menu Bar:** Bosh panel — *File, Edit, View, Help* kabi bo‘limlarni ushlab turadi.
- **Menu:** Har bir bo‘limning ochiladigan ro‘yxati (masalan, *File* ichida *Open, Save, Exit*).
- **Action (QAction):** Menyudagi bitta band (buyruq). U bosilganda signal (*triggered*) yuboradi.
- **Submenu:** Ichki menu (masalan, *View → Zoom → In/Out*).
- **Separator:** Guruhlash uchun ajratgich chiziq.

3) Muhim xususiyatlar

- **Ierarxiya:** Menu → Submenu → Action zanjiri bilan murakkab tuzilma yasash mumkin.
- **Holatlar:** *enabled/disabled*, **checkable** (belgilanadigan), **exclusive** (guruh ichida bittasi tanlanadi).
- **Klaviatura qisqartmalari (Shortcut):** Masalan, *Ctrl+O, Ctrl+Q*.
- **Status/Tool tip:** Har bir action’ga **statusTip** (status bar’da ko‘rinadi) va **toolTip** berish mumkin.
- **Dinamik boshqaruv:** Dastur ishlayotganda menyuga band qo‘shish/olib tashlash, matnni yoki belgini o‘zgartirish.
- **Signal-slot:** *triggered, toggled* kabi signallarni funksiyalarga ulash.
- **Platform moslashuvi:** Windows/Linux’da yuqorida, macOS’da system menubar’da ko‘rinishi mumkin.
- **Kontext menu** (o‘ng tugma menu): Muayyan vidjet uchun kontekstli buyruqlar (masalan, matn maydoni ustida “Copy/Paste”).
- **UX tavsiyalar:**
 - Buyruqlarni mantiqan guruhlang; ajratgichdan foydalaning.
 - Muhim bandlarga qulay **shortcut** bering.
 - “Destructive” (masalan, *Delete/Exit*) amallarni ajratib ko‘rsating.
 - “About”, “Help” kabi ma’lumotlar odatda **Help** menyusida bo‘ladi.
 - Tanlov-stillari uchun **checkable** va **exclusive group** ishlating.

Misollar

1) Qt Designer'da .ui faylini tayyorlash (juda sodda)

1. **Qt Designer** → **Main Window** shablonini tanlang.
2. Yuqoridagi **Menu Bar** ichida bo'limlar yarating:
 - o &File → Say Hello, Clear, separator, Exit
 - o &Help → About
3. Har bir bandga **objectName** bering (Properties → *objectName*):
 - o actionSayHello, actionClear, actionExit, actionAbout
4. Markazga **QLabel** qo'ying:
 - o *objectName*: labelMsg
 - o *alignment*: **AlignCenter**
 - o *text*: (bo'sh qoldirsangiz ham bo'ladi)
5. Faylni **main_window.ui** nomi bilan saqlang.

Shu bilan UI tayyor. Endi kichik Python kod bilan ishlatamiz.

2) Python kodi (uic.loadUi bilan yechim)

main_window.ui bilan bir papkada quyidagi main.py faylini yarating:

```
import sys
from PyQt5 import QtWidgets, uic
from PyQt5.QtWidgets import QMessageBox

class MainWindow(QtWidgets.QMainWindow):
    def __init__(self):
        super().__init__()
        uic.loadUi("main_window.ui", self) # .ui faylni yuklaymiz

        # Menu action'larini ulash (Designer'dagi objectName'lar bilan)
        self.actionSayHello.triggered.connect(self.say_hello)
        self.actionClear.triggered.connect(self.clear_text)
        self.actionExit.triggered.connect(self.close) # tayyor slot
        self.actionAbout.triggered.connect(self.show_about)

        # Dastlabki holat
        self.labelMsg.setText("Menyudan biror amalni tanlang 😊")
        self.statusBar().showMessage("Tayyor")

        # --- Slotlar (funksiyalar)
        def say_hello(self):
            self.labelMsg.setText("Salom, Menyu!")
            self.statusBar().showMessage("Hello bosildi")

        def clear_text(self):
            self.labelMsg.clear()
            self.statusBar().showMessage("Matn tozalandi")

        def show_about(self):
```

```
        QMessageBox.information(  
            self,  
            "About",  
            "Eng oddiy menyu misoli: File → Say Hello / Clear / Exit, Help → About"  
        )  
  
if __name__ == "__main__":  
    app = QtWidgets.QApplication(sys.argv)  
    w = MainWindow()  
    w.setWindowTitle("Oddiy Menyu – PyQt misoli")  
    w.resize(420, 260)  
    w.show()  
    sys.exit(app.exec_())
```

Nima o‘rgandik?

- **Menu Bar** ichida **Action** lar yaratdik va triggered signalini funksiyalarga uladik.
- Say Hello → markazdagi QLabel matnini o‘zgartiradi.
- Clear → matnni tozalaydi.
- Exit → dasturdan chiqadi.
- About → QMessageBox ko‘rsatadi.